

Informatik - Programmieren mit Java - Handout

1 Arbeiten mit der Konsole

Die *Konsole* ist ein Werkzeug zur Steuerung eines Computers.

1.1 Konsole öffnen

Unter Windows können wir die Konsole unter anderem so öffnen:

- Windows + R drücken, cmd eingeben, Enter drücken
- In der Windows-Suche (unten links) nach cmd oder Eingabeaufforderung suchen, auf Eingabeaufforderung klicken

1.2 Konsole bedienen

Öffnen wir die Konsole, sehen wir in etwa Folgendes:

```
1 C:\Users\Lernen>
```

Der Teil vor dem `>` ist der Pfad, „an dem wir uns befinden“ - das sogenannte *Arbeitsverzeichnis*.

Wir haben 2 Konsolen-Befehle kennengelernt:

- `cd` (*change directory*) - wechselt das Arbeitsverzeichnis.
Anwendung:

```
1 cd Unterordner
2 cd C:\Users\Lernen\Desktop\MeinOrdner
3 cd ..
```

(Zeile 1: Wechsle zu einem *Unterordner* im aktuellen Ordner)

(Zeile 2: Wechsle zum angegebenen Pfad)

(Zeile 3: Wechsle zum *Überordner*)

- `dir` (*directory*) - zeigt alle Dateien und Ordner im aktuellen Arbeitsverzeichnis an

2 Java installieren

Bevor wir mit Java arbeiten können, müssen wir das *JDK* (Java-Development-Kit) installieren. Am einfachsten geht das über die Konsole mit diesem Befehl:

```
1 winget install EclipseAdoptium.Temurin.17.JDK
```

Schließen wir nach erfolgter Installation die Konsole und öffnen sie erneut, können wir mit dem Befehl `java` prüfen, ob die Installation erfolgreich war. War Sie es nicht, sehen wir einen Fehler, in etwa „Der Befehl gibt es nicht“ - ansonsten einen längeren Hilfetext von Java selbst.

3 Java - absolute Grundlagen

3.1 Java-Programm ausführen

Wir haben die Grundlagen betrachtet, wie man mit der Programmiersprache Java arbeitet. Noch einmal zusammengefasst:

- Wir schreiben *Programmcode* (oder einfach *Code*) in einer Datei, z.B. TollesProgramm.java
- Wir *kompilieren* das Programm mit `javac TollesProgramm.java`
- Wir führen es aus mit `java TollesProgramm`

Der Befehl `javac` erzeugt eine Datei `TollesProgramm.class`, wenn alles funktioniert hat. Ansonsten bekommen wir eine *Fehlermeldung*. Letztere beschreibt mal mehr mal weniger genau, wo wir einen Fehler gemacht haben.

Beachte: Damit die Befehle `javac` und `java` funktionieren, müssen wir im korrekten Arbeitsverzeichnis sein (nämlich dort, wo sich unsere Datei `TollesProgramm.java` befindet).

3.2 Minimales Java Programm

Zuletzt haben wir ein minimales Java-Programm betrachtet. Dieses sieht wie folgt aus:

```
1 public class Test {  
2     public static void main(String[] args) {  
3     }  
4 }
```

Schnipsel 1: Minimales Java Programm

Dieses Programm tut gar nichts. Diese 4 Zeilen stellen das „Grundgerüst“ eines Java-Programms dar.

Erste neue Kleinigkeit: mit `//` (doppel-slash) können wir *Kommentare* in unserem Code schreiben. Alles innerhalb einer Zeile, was nach `//` kommt, wird zu einem Kommentar. Ein Kommentar wird von Java komplett ignoriert, und taucht im kompilierten Code (der `.class`-Datei) nicht auf.

Wir nutzen Kommentare meistens, um anderen Lesern des Codes (oder unserem zukünftigen Ich) zu erläutern, WARUM bestimmte Dinge im Code so sind wie sie sind (WAS passiert brauchen wir nicht zu erklären, das sagt ja der Programmcode).

Die folgenden Zeilen sind kompiliert also exakt das selbe Programm wie in Schnipsel 1:

```
1 // Ich bin nur ein Kommentar und habe sonst keine Funktion  
2 public class Test { // Ich bin auch ein Kommentar (der Teil vor mir aber nicht)  
3     public static void main(String[] args) {  
4         // Ich bin noch ein Kommentar  
5         // Wieder ein Kommentar  
6     }  
7     // Ich stehe hier nur so  
8 }
```

Schnipsel 2: Kommentare

3.3 Der main-Block

Wir können noch nicht jedes Detail des Grundgerüsts klären. Es reicht aber zunächst zu wissen, dass unser Programmcode in den inneren Block, den „main-Block“ gehört:

```
1 public class Test {
2     public static void main(String[] args) {
3         // Das hier ist der main-Block unseres Programms.
4         // Hier schreiben wir (vorerst) unseren gesamten Programmcode rein
5     }
6 }
```

Schnipsel 3: main-Block

4 Java - Grundlagen

Die Befehle, die wir in den main-Block schreiben, werden nacheinander von oben nach unten abgearbeitet.

Wir halten einmal die ersten wichtigen Java Befehle fest. Jeder Befehl wird mit einem ; (*Semikolon*) beendet.

4.1 Ausgabe

Mit `System.out.println` können wir unser Programm etwas auf der Konsole ausgeben lassen. Auszugebenden Text setzen wir dabei in Anführungszeichen (Erklärung im Abschnitt „Strings“). Beispiel:

```
1 public class Test {
2     public static void main(String[] args) {
3         System.out.println("Ich bin ein interessanter Text");
4         System.out.println("42");
5
6         // Die nächste Zeile produziert die selbe Ausgabe wie die vorherige, die Zahl wird einfach zu
7         // Text umgewandelt:
8         System.out.println(42);
9     }
10 }
```

Schnipsel 4: Ausgabe auf der Konsole

4.2 Variablen

Eine Variable ist ein *Platzhalter*. Eine Variable hat stets einen

- *Datentyp* (oder einfach Typ),
- einen *Namen*,
- und einen *Wert*.

Der Datentyp gibt an WAS FÜR EINE ART von Werten in der Variable gespeichert werden kann. Den Namen nutzen wir, um auf den Wert der Variablen zuzugreifen oder allgemein „etwas mit ihr zu tun“.

Beispiel:

```
1 public class Test {
2     public static void main(String[] args) {
3         int anzahlBananen = 17;
4         System.out.println(anzahlBananen);
5     }
6 }
```

Schnipsel 5: Variable

Hier erstellen wir eine Variable vom Typ `int` (das steht für *Integer*, englisch für Ganzzahl) und weisen ihr den Wert 17 zu.

Je nach Datentyp können wir verschiedene Dinge mit Variablen anstellen.

Zahlen können wir zum Beispiel mit `+` addieren:

```
1 public class Test {
2     public static void main(String[] args) {
3         int a = 15;
4         int b = 21;
5         int summe = a + b;
6         System.out.println(summe);
7     }
8 }
```

Schnipsel 6: Summe zweier Ganzzahlen

und genauso subtrahieren (`-`), multiplizieren (`*`) oder dividieren (`/`).

Java kann auch komplexere Terme interpretieren:

```
1 public class Test {
2     public static void main(String[] args) {
3         int a = 4;
4         int b = 5;
5         int ergebnis1 = (a + b) * (a + b);
6         int ergebnis2 = a * a + 2 * b * a + b * b;
7
8         // In Andenken an die binomische Formel hoffen wir mal dass hier zweimal das selbe rauskommt:
9         System.out.println(ergebnis1);
10        System.out.println(ergebnis2);
11    }
12 }
```

Schnipsel 7: binomische Formel

4.3 Einer Variablen einen neuen Wert zuweisen

Mit dem Befehl `int x = 5` tun wir eigentlich 2 Dinge auf einmal:

- Wir *deklarieren* die Variable `x` vom Typ `int` (wir „sagen dass es sie gibt“, englisch „to declare“),
- und wir weisen `x` den Wert 5 zu.

Wenn wir den Wert einer Variablen nachträglich ändern wollen, nutzen wir dieselbe Schreibweise, aber ohne den Typ zu wiederholen:

```
1 public class Test {
2     public static void main(String[] args) {
3         int x = 5;
4         System.out.println(x);
5         x = -3;
6         System.out.println(x);
7     }
8 }
```

Schnipsel 8: Wert einer Variablen ändern

Variablenamen müssen eindeutig sein (innerhalb eines „scopes“, dazu später mehr). Schreiben wir `int x = 5` und später `int x = -3`, denkt der Compiler, dass wir zwei verschiedene Variablen mit dem Namen `x` haben wollen - das führt zu einem Fehler.

4.4 Weitere Datentypen

Weitere Datentypen sind:

- `double` (für Kommazahlen, z.B. `double x = 0.5543`)
- `boolean` (für Wahrheitswerte, z.B. `boolean T = true`)
- `String` (für Zeichenketten, z.B. `String text = "Bananenkuchen"`)

Beachte: `String` gehört eigentlich zu einer anderen Kategorie als `int`, `double`, `boolean`. Letztere sind sogenannte *primitive Datentypen*, `String` dagegen **nicht** - aber dazu später mehr.

Anmerkung: Wir können auch `String`'s addieren:

```
1 public class Test {
2     public static void main(String[] args) {
3         String teil1 = "Ciao";
4         String teil2 = "Kakao";
5         System.out.println(teil1 + " " + teil2);
6     }
7 }
```

Schnipsel 9: Strings addieren

Wir können Strings sogar mit beliebigen anderen Variablen addieren:

```
1 public class Test {
2     public static void main(String[] args) {
3         int x = 18;
4         System.out.println("Hier liegen " + x + " Maracujas");
5     }
6 }
```

Schnipsel 10: String + Zahl

Die Summanden, die nicht vom Typ `String` sind, werden dabei zu Strings umgewandelt.

4.5 Bedingungen und Kontrollstrukturen

Unser Programm wird zwar von oben nach unten, Befehl für Befehl abgearbeitet - dennoch wollen wir manchmal dass es sich nicht immer exakt gleich verhält.

Eine Art dies zu tun, ist, den Programmfluss an *Bedingungen* zu knüpfen - einfach ausgedrückt: „Wenn das, dann... sonst etwas anderes“.

Um zu überprüfen „ob etwas gilt“ brauchen wir Wahrheitswerte (englisch: boolean). Eine Variable vom typ `boolean` kann nur genau zwei Werte haben: `true` oder `false`. *True* ist englisch für „wahr“, *False* für „falsch“.

Wir können so einen Wahrheitswert zum Beispiel durch den Vergleich von Zahlenwerten vergleichen:

```
1 public class Test {
2     public static void main(String[] args) {
3         int x = 4;
4         int y = 23494;
5
6         boolean xKleinerY = x < y;
7         boolean yKleinerX = y < x;
8         boolean xGroesserY = x > y;
9         boolean yGroesserX = y > x;
10        boolean xGleichY = x == y;
11        boolean xUngleichY = x != y;
12
13        System.out.println(xKleinerY);
14        System.out.println(yKleinerX);
15        // ...
16    }
17 }
```

Schnipsel 11: Wahrheitswerte

Wahrheitswerte können wir nutzen, um bestimmte Teile des Programms optional ausführen zu lassen - nämlich nur dann wenn eine Bedingung wahr ist.

Das geht wie folgt:

```
1 public class Test {
2     public static void main(String[] args) {
3         int x = 5;
4         double y = 3.4;
5
6         // Der Code innerhalb des if-Blocks wird nur ausgefuehrt, wenn das Resultat von "x > y" true
6         // ist:
7         if(x > y) {
8             System.out.println("x ist groesser als y!!!!");
9         }
10    }
11 }
```

Schnipsel 12: Variable

Diese Struktur nennt man *if-Abfrage* oder *if-Statement* - *if* bedeutet auf englisch „wenn“.

Ein `if` können wir zudem mit einem `else` (englisch für sonst) kombinieren:

```
1 public class Test {
2     public static void main(String[] args) {
3         int x = 5;
4         double y = 3.4;
5     }
```

```

6      // Der Code innerhalb des if-Blocks wird nur ausgefuehrt, wenn das Resultat von "x > y" true
      ist:
7      if(x > y) {
8          System.out.println("x ist groesser als y!!!!");
9      } else {
10         // Der code in diesem else Block wird nur "sonst" ausgefuehrt, also wenn die Bedingung im
            if(...) NICHT wahr ist
11         System.out.println("x ist anscheinend nicht groesser als y");
12     }
13 }
14 }

```

Schnipsel 13: Variable

Konkretes Beispiel: Wir wollen entscheiden, ob ein Kreis mit dem angegebenen Radius größer oder kleiner als ein Grenzwert ist.

```

1 public class Test {
2     public static void main(String[] args) {
3         double PI = 3.141
4
5         double radius = 13;
6         double grenzwert = 30;
7
8         double flaeche = PI * radius * radius;
9
10        if(flaeche > grenzwert) {
11            System.out.println("Die Flaeche ist " + flaeche + ", groesser als der Grenzwert!");
12        } else {
13            System.out.println("Die Flaeche ist " + flaeche + ", das ist ok.");
14        }
15    }
16 }

```

Schnipsel 14: Kreisfläche